

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧ. ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
Τομέας Τεχνολογίας Πληροφορικής και Υπολογιστών
Εργαστήριο Μικροϋπολογιστών και Ψηφιακών Συστημάτων

Σχεδιασμός Ενσωματωμένων Συστημάτων
9ο Εξάμηνο ΗΜΜΥ

1η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ

ΒΕΛΤΙΣΤΟΠΟΙΗΣΗ ΑΛΓΟΡΙΘΜΩΝ ΓΙΑ ΧΑΜΗΛΗ ΚΑΤΑΝΑΛΩΣΗ ΕΝΕΡΓΕΙΑΣ ΚΑΙ ΥΨΗΛΗ ΑΠΟΔΟΣΗ

Αναφορά

Ονοματεπώνυμο: **Ζακυνθινός Ιάσων – Δανιήλ**
Α.Μ.: _____
Ακαδημαϊκό έτος: 2026–2027

ΑΝΤΙΓΡΑΦΟ ΕΡΓΑΣΤΗΡΙΟΥ – ΟΧΙ ΓΙΑ ΠΑΡΑΔΟΣΗ. Οι **μπλε** τιμές του Ζητούμενου 1 είναι **τιμές αναφοράς** από παλαιότερη αναφορά για την ίδια άσκηση (jimmichalako/embedded-systems-ntua (2024–25)), στην ίδια πλακέτα στα 240 MHz. **Δεν είναι δικές μας μετρήσεις.** Είναι άμεσα συγκρίσιμες για τον αρχικό κώδικα (v0)· στα B, $B_x \times B_y$ συγκρίνεται μόνο η τάση, αφού ο δικός τους βελτιστοποιημένος κώδικας είναι διαφορετικός. Οι **μωβ** τιμές ($\approx$) για τα v1–v4 είναι **εκτιμήσεις** για τον δικό μας κώδικα από προσομοίωση Cortex-M4 (`tools/m4estimate`), βαθμονομημένη στις παραπάνω μετρήσεις· ακρίβεια περίπου ±15 %.

Ζητούμενο 1: Ο αλγόριθμος PHODS στην πλακέτα Renesas S7G2

Ο κώδικας του ζητούμενου εκτελείται στην πλακέτα SK-S7G2 (μικροελεγκτής R7FS7G27H3A01CFC) μέσω του e^{2}studio, με απενεργοποιημένες τις βελτιστοποιήσεις του compiler (`-O0`). Οι διαστάσεις του προβλήματος είναι αυτές του σκελετού της άσκησης: $N = M = 10$ και $B = 5$. Όλες οι εκδόσεις του αλγορίθμου που περιγράφονται παρακάτω μεταγλωττίζονται στο ίδιο πρόγραμμα, το οποίο στη μία και μοναδική εκτέλεσή του τις χρονομετρά όλες. Ο πλήρης κώδικας βρίσκεται στο αποθετήριο που αναφέρεται στο Παράρτημα.

Εκκρεμεί (μετρήσεις εργαστηρίου): οι αριθμητικές τιμές του Ζητούμενου 1 (πίνακες με «–») συμπληρώνονται αυτόματα από τις μετρήσεις στην πλακέτα· τα σχόλια που εξαρτώνται από τα αποτελέσματα γράφονται μετά.

1. Χαρακτηριστικά της πλακέτας

Τα μεγέθη των μνημών προκύπτουν από το datasheet της οικογένειας S7G2 (R01DS0262EU0150, Rev. 1.50) και συνοψίζονται στον Πίνακα 1.

Πίνακας 1. Μνήμες του μικροελεγκτή R7FS7G27H3A01CFC.

Μνήμη	Μέγεθος	Παρατηρήσεις
Code Flash	4 MB	Κώδικας και όλα τα δεδομένα `const` (άρα και τα frames του `images.h`). Zero wait states έως τα 80 MHz.
Data Flash	64 KB	Μη πτητικά δεδομένα χρήστη, 125 000 κύκλοι εγγραφής/διαγραφής.
SRAM	640 KB	Τα πρώτα 32 KB της SRAM0 με Double-bit Error Detection, τα υπόλοιπα 608 KB με parity. Εδώ βρίσκονται stack, καθολικές και τοπικές μεταβλητές.
Standby SRAM	8 KB	Διατηρεί το περιεχόμενό της στο Deep Software Standby mode.

Κρυφές μνήμες. Ο πυρήνας ARM Cortex-M4 δεν διαθέτει δικές του instruction ή data caches. Η μόνη cache στο σύστημα είναι η **Flash Cache (FCACHE)**, η οποία βρίσκεται μπροστά από τη code flash. Η flash λειτουργεί χωρίς wait states μόνο μέχρι τα 80 MHz, ενώ ο επεξεργαστής φτάνει τα 240 MHz· χωρίς την FCACHE κάθε ανάγνωση από τη flash θα κόστιζε επιπλέον κύκλους αναμονής. Η FCACHE κρατά περιεχόμενο της flash που διαβάστηκε πρόσφατα, ώστε οι επαναλαμβανόμενες αναγνώσεις (εντολές βρόχων, πίνακες `const`) να εξυπηρετούνται γρήγορα. Οι προσβάσεις στη SRAM δεν περνούν από κάποια cache, αλλά η on-chip SRAM είναι ούτως ή άλλως γρήγορη. Το Memory Mirror Function (MMF) και τα Memory Protection Units που αναφέρει το datasheet δεν είναι κρυφές μνήμες.

Συχνότητα ρολογιού. Η μέγιστη συχνότητα του πυρήνα (ICLK) είναι 240 MHz και τροφοδοτείται από τον κύριο ταλαντωτή (8–24 MHz) μέσω PLL. Η πραγματική συχνότητα με την οποία προγραμματίζεται η πλακέτα διαβάστηκε από τη μεταβλητή `SystemCoreClock` κατά την εκτέλεση: **240 MHz**. Η ίδια τιμή χρησιμοποιείται για τη μετατροπή των κύκλων σε χρόνο.

2. Μέτρηση του χρόνου εκτέλεσης

Ο χρόνος μετράται με τον μετρητή κύκλων `DWT->CYCCNT` του Cortex-M4. Μετά την ενεργοποίησή του (κώδικας της εκφώνησης), ο μετρητής διαβάζεται αμέσως πριν και αμέσως μετά από κάθε κλήση της `phods_motion_estimation`, ώστε να μετράται μόνο η ίδια η συνάρτηση:

```
uint32_t t0 = DWT->CYCCNT;
phods_motion_estimation(current, previous, vectors_x, vectors_y);
uint32_t cycles = DWT->CYCCNT - t0;     /* unsigned: σωστό και με overflow */
float    time_us = cycles / (SystemCoreClock / 1e6f);
```

Η συνάρτηση εκτελείται 10 φορές και κρατάμε ελάχιστο, μέγιστο και μέσο χρόνο. Τα αποτελέσματα αποθηκεύονται σε έναν καθολικό πίνακα `volatile` και διαβάζονται από τον debugger. Για τον αρχικό κώδικα:

Πίνακας 2. Χρόνος εκτέλεσης του αρχικού κώδικα ($B = 5$, 10 εκτελέσεις).

	Ελάχιστος	Μέγιστος	Μέσος
Κύκλοι	437 069	437 070	437 069
Χρόνος (μs)	1821,12	1821,13	1821,12

Εκκρεμεί (μετρήσεις εργαστηρίου): σχόλιο για τη διασπορά των 10 μετρήσεων (αναμένεται min ≈ max, αφού η είσοδος είναι σταθερή και δεν τρέχει τίποτε άλλο στον επεξεργαστή).

3. Μετασχηματισμοί του κώδικα

Οι μετασχηματισμοί εφαρμόστηκαν σταδιακά και κάθε στάδιο κρατήθηκε ως ξεχωριστή συνάρτηση, ώστε να μετρηθεί η συνεισφορά του καθενός. Όλες οι εκδόσεις δίνουν ακριβώς τα ίδια motion vectors με τον αρχικό κώδικα. Αυτό ελέγχεται τόσο στην πλακέτα, όπου κάθε έκδοση συγκρίνεται με την αρχική, όσο και σε υπολογιστή, όπου όλες οι εκδόσεις και όλα τα μεγέθη block συγκρίθηκαν με την αρχική υλοποίηση σε 2000 τυχαία ζεύγη frames.

v1 – Loop-invariant code motion. Στον αρχικό κώδικα οι εκφράσεις `B*x + vectors_x[x][y] + i + k` υπολογίζονται ξανά για κάθε pixel, και ο πίνακας `vectors_x` διαβάζεται από τη μνήμη περίπου τέσσερις φορές ανά pixel. Η θέση του block (`B*x`, `B*y`) υπολογίζεται πλέον μία φορά ανά block και το τρέχον motion vector κρατιέται σε τοπικές μεταβλητές, που γράφονται στον πίνακα εξόδου μόνο στο τέλος. Ο έλεγχος ορίων για τη γραμμή εξαρτάται μόνο από το `k`, οπότε μεταφέρθηκε έξω από τον εσωτερικό βρόχο. Ο ξεχωριστός βρόχος αρχικοποίησης των διανυσμάτων έγινε περιττός.

v2 – Αφαίρεση των ελέγχων ορίων από τον εσωτερικό βρόχο. Για κάθε υποψήφιο block υπολογίζεται μία φορά το διάστημα γραμμών $[k_{\min}, k_{\max})$ και στηλών $[l_{\min}, l_{\max})$ που πέφτει μέσα στο frame. Μέσα σε αυτό ο βρόχος κάνει μόνο αφαίρεση, απόλυτη τιμή και πρόσθεση, χωρίς καμία σύγκριση:

```
for (l = lmin; l < lmax; l++) { d = cp[l] - *pp++; sum += (d < 0) ? -d : d; }
```

Τα pixel εκτός frame συνεισφέρουν $|p_1|$, αφού εκεί $p_2 = 0$. Για ολόκληρες γραμμές εκτός frame χρησιμοποιείται το άθροισμα $\sum |p_1|$ της γραμμής, που υπολογίζεται μία φορά ανά block.

v3 – Επαναχρησιμοποίηση του κεντρικού υποψηφίου. Σε κάθε βήμα η αναζήτηση στον άξονα x και η αναζήτηση στον άξονα y εξετάζουν για $i = 0$ το ίδιο block, στη θέση του τρέχοντος διανύσματος. Το SAD του υπολογίζεται πλέον μία φορά και χρησιμοποιείται και στις δύο αναζητήσεις, άρα 5 αντί για 6 υπολογισμοί SAD ανά βήμα (περίπου 17% λιγότερη δουλειά). Η σειρά των συγκρίσεων και η αυστηρή ανισότητα διατηρούνται, ώστε οι ισοπαλίες να λύνονται όπως στον αρχικό κώδικα.

v4 – Αντιγραφή του τρέχοντος block στη RAM. Τα frames είναι `const` και επομένως βρίσκονται στη flash. Το τρέχον block διαβάζεται 15 φορές (3 βήματα × 5 SAD), οπότε δοκιμάστηκε να αντιγράφεται μία φορά ανά block σε τοπικό buffer στη SRAM.

Πίνακας 3. Χρόνος εκτέλεσης ανά στάδιο βελτιστοποίησης ($B = 5$, 10 εκτελέσεις).

	Μετασχηματισμός	Ελάχ. (μs)	Μέγ. (μs)	Μέσος (μs)	Επιτάχυνση
v0	αρχικός κώδικας	1821,12	1821,13	1821,12	1,00×
v1	loop-invariant code motion	εκτ.	εκτ.	≈ 710	≈ 2,6×
v2	+ χωρίς ελέγχους ορίων	εκτ.	εκτ.	≈ 540	≈ 3,4×
v3	+ κοινός κεντρικός υποψήφιος	εκτ.	εκτ.	≈ 449	≈ 4,1×
v4	+ block σε buffer στη RAM	εκτ.	εκτ.	≈ 468	≈ 3,9×

Αναφορά σύγκρισης: ο τελικός βελτιστοποιημένος κώδικας της παλαιότερης αναφοράς ($B = 5$, διαφορετικοί μετασχηματισμοί) έτρεχε σε 738,00 μs, δηλαδή περίπου 2,5× ταχύτερα από τον αρχικό. Εκτιμώμενα εύρη για τον δικό μας κώδικα (3 μοντέλα): v1 674–770, v2 516–599, v3 431–497, v4 450–515 μs. Ο v0 μας εκτελεί τις ίδιες ακριβώς εντολές με τον δικό τους αρχικό (193 811 έναντι 193 799), άρα το ≈1821 μs πρέπει να το δεις σχεδόν ακριβώς. Αν το v4 βγει πιο αργό από το v3, είναι αναμενόμενο (η εκτίμηση δίνει ∼4 % πιο αργό).

Εκκρεμεί (μετρήσεις εργαστηρίου): σχολιασμός της συνεισφοράς κάθε σταδίου και σύγκριση με το ερώτημα 2. Ειδικά για το v4 (υπόδειξη της εκφώνησης): αν *δεν* βελτιώνει, εξήγηση ότι τα `const` δεδομένα βρίσκονται στη flash πίσω από την FCACHE, οπότε οι επαναλαμβανόμενες αναγνώσεις είναι ήδη γρήγορες, ενώ η αντιγραφή προσθέτει κόστος και με `-O0` και ο τοπικός buffer είναι μνήμη (stack) και όχι καταχωρητές. Αν βελτιώνει, εξήγηση ότι οι αναγνώσεις της flash στα 240 MHz κοστίζουν wait states όταν αστοχεί η FCACHE, ενώ η SRAM όχι. Επίσης: αν το v4 είναι ταχύτερο, τα ερωτήματα 4–5 ξανατρέχουν με αυτό.

4. Design space exploration για το μέγεθος block B

Το B πρέπει να διαιρεί και το N και το M, άρα οι υποψήφιες τιμές είναι $B \in \{1, 2, 5, 10\}$. Για κάθε τιμή μεταγλωττίζεται ξεχωριστή έκδοση του βελτιστοποιημένου κώδικα (v3), με το B ως σταθερά χρόνου μεταγλώττισης όπως στον αρχικό κώδικα, και εκτελείται 10 φορές.

Πίνακας 4. Χρόνος εκτέλεσης ανά τετραγωνικό μέγεθος block.

B	Πλήθος blocks	Ελάχ. (μs)	Μέγ. (μs)	Μέσος (μs)
1	100	1236,79	1236,80	1236,79
2	25	848,14	848,15	848,14
5	4	738,01	738,02	738,01
10	1	681,57	681,58	681,57

Η καλύτερη απόδοση επιτυγχάνεται για $B = 10$ (μέσος χρόνος 681,57 μs). Ο συνολικός αριθμός πράξεων ανά pixel είναι περίπου ίδιος για κάθε B (3 βήματα × 5 SAD × B^2 pixels × $(N/B)^2$ blocks $= 15NM$). Αυτό που αλλάζει είναι το σταθερό κόστος ανά block (υπολογισμός ορίων, αρχικοποιήσεις, κλήσεις της συνάρτησης SAD), που πολλαπλασιάζεται με τον αριθμό των blocks, και το μήκος του εσωτερικού βρόχου. Με μεγάλο B ο εσωτερικός βρόχος διατρέχει περισσότερα συνεχόμενα στοιχεία μιας γραμμής, και αυτό αξιοποιεί καλύτερα τις γραμμές της FCACHE.

Οι μπλε τιμές είναι του δικού τους κώδικα. Εκτίμηση για τον δικό μας (v3) κώδικα: $B = 1/2/5/10 \rightarrow$ 1981 / 815 / 449 / 365 μs, δηλαδή ίδια τάση ($B = 10$ ταχύτερο) αλλά περίπου 40 % χαμηλότερα στα μεγάλα B.

Εκκρεμεί (μετρήσεις εργαστηρίου): επιβεβαίωση με τις μετρήσεις και σχέση με το ερώτημα 1: σύνδεση της βέλτιστης τιμής με το μέγεθος γραμμής της FCACHE / την ταχύτητα της flash και με το ότι για $B = 10$ όλο το frame είναι ένα block.

5. Design space exploration για ορθογώνιο block $B_x \times B_y$

Για ορθογώνιο block το B_x (γραμμές) πρέπει να διαιρεί το N και το B_y (στήλες) το M, δηλαδή 16 συνδυασμοί. Ο Πίνακας 5 δίνει τον μέσο χρόνο για καθέναν.

Πίνακας 5. Μέσος χρόνος εκτέλεσης (μs) για κάθε ζεύγος $B_x \times B_y$.

$B_x \backslash B_y$	1	2	5	10
1	1236,79	931,04	769,12	719,62
2	1005,96	848,14	737,77	684,94
5	864,46	773,76	738,01	700,06
10	834,16	747,36	711,38	681,57

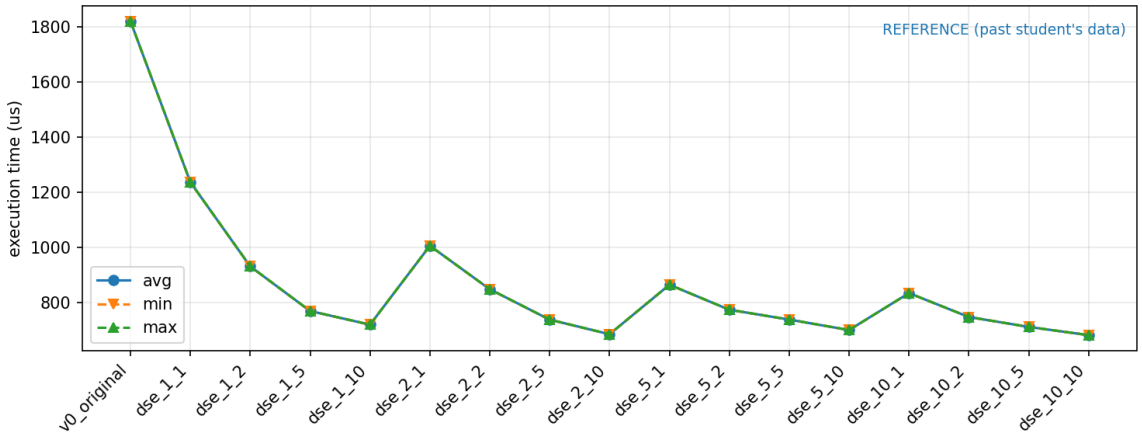
Το βέλτιστο ζεύγος είναι $B_x = 10$, $B_y = 10$ (681,57 μs).

Εκτίμηση για τον δικό μας κώδικα (μέσος, μs), γραμμές $B_x = 1, 2, 5, 10$, στήλες $B_y = 1, 2, 5, 10$: 1981/1130/644/475; 1342/815/522/413; 973/633/449/377; 852/572/425/365. Αναμενόμενο βέλτιστο: 10×10.

Ευρετικός περιορισμός της αναζήτησης. Ο χρόνος καθορίζεται κυρίως από τον αριθμό των blocks $(N/B_x)(M/B_y)$, αφού η δουλειά ανά pixel είναι σταθερή, και δευτερευόντως από το μήκος του εσωτερικού βρόχου, που είναι B_y συνεχόμενα στοιχεία στη μνήμη. Επομένως αρκεί να εξεταστούν οι μεγάλες τιμές, ξεκινώντας από το βέλτιστο τετραγωνικό B του ερωτήματος 4 και τους γείτονές του, με προτεραιότητα σε μεγαλύτερο B_y από B_x. Οι συνδυασμοί με $B_x = 1$ ή $B_y = 1$ μπορούν να απορριφθούν από την αρχή.

Εκκρεμεί (μετρήσεις εργαστηρίου): επιβεβαίωση της ευρετικής με τον Πίνακα 5 (π.χ. ότι για ίδιο πλήθος blocks το μεγαλύτερο B_y είναι ταχύτερο).

Συνολικό διάγραμμα



Σχήμα 1. Ελάχιστος, μέσος και μέγιστος χρόνος εκτέλεσης για όλες τις εκδόσεις (ερωτήματα 2–5).

Εκκρεμεί (μετρήσεις εργαστηρίου): συνολική σύγκριση και ανάλυση των αποτελεσμάτων του Σχήματος 1.

Ζητούμενο 2: Design space exploration με τον προσομοιωτή gem5

Οι προσομοιώσεις έγιναν με το έτοιμο Docker image του μαθήματος (`aferikoglou/gem5built:v1.0`, gem5 23.0) και τα configuration scripts του `learning_gem5/part1`. Το σύστημα αποτελείται από έναν επεξεργαστή `X86TimingSimpleCPU` στο 1 GHz και μνήμη DDR3-1600. Στο `two_level.py` οι L1 caches είναι 2-way set associative με latency 2 κύκλων και η L2 είναι 8-way με latency 20 κύκλων. Όλες έχουν γραμμή 64 bytes. Τα `tables*.exe` μεταγλωττίστηκαν με `gcc -O0`. Το `simple.py` τροποποιήθηκε ώστε να δέχεται το εκτελέσιμο ως όρισμα, όπως ζητά η εκφώνηση.

Η εφαρμογή `tables.c` δεσμεύει τέσσερις πίνακες `double` των $N = 100\,000$ στοιχείων (800 KB ο καθένας, 3,2 MB συνολικά). Τους αρχικοποιεί σε έναν βρόχο και στη συνέχεια υπολογίζει `y[i] += a1*x1[i] + a2*x2[i] + a3*x3[i]`. Πρόκειται για καθαρό streaming: κάθε στοιχείο χρησιμοποιείται μία φορά ανά βρόχο.

1. Αρχιτεκτονική χωρίς caches

Με το `simple.py`, όπου ο επεξεργαστής συνδέεται απευθείας στο memory bus, η εκτέλεση απαιτεί **857 689 669 κύκλους** (`system.cpu.numCycles`).

2. Αρχιτεκτονική με L1 και L2 caches

Με L1I = 8 KB, L1D = 8 KB και L2 = 128 KB η εκτέλεση απαιτεί **59 787 087 κύκλους**, δηλαδή 14,3 φορές λιγότερους. Χωρίς caches κάθε ανάκληση εντολής και κάθε load/store φτάνει μέχρι τη DRAM. Το πρόγραμμα εκτελεί περίπου 8,2 εκατομμύρια εντολές με 11,6 εκατομμύρια ανακλήσεις εντολών και 4,4 εκατομμύρια προσβάσεις δεδομένων, άρα ο χρόνος κυριαρχείται από την καθυστέρηση της κύριας μνήμης.

Με τις caches η εικόνα αλλάζει ριζικά. Ο κώδικας αποτελείται από λίγους μικρούς βρόχους και χωράει άνετα στην L1I, η οποία έχει miss rate μόλις 0,016%. Οι προσβάσεις δεδομένων εξυπηρετούνται κυρίως από την L1D ή την L2. Όπως φαίνεται στο επόμενο ερώτημα, με L1D = 8 KB τα L1D misses είναι πολλά, όμως τα δεδομένα βρίσκονται ακόμη στην L2 (L2 miss rate 12,6%) και κοστίζουν περίπου 20 κύκλους

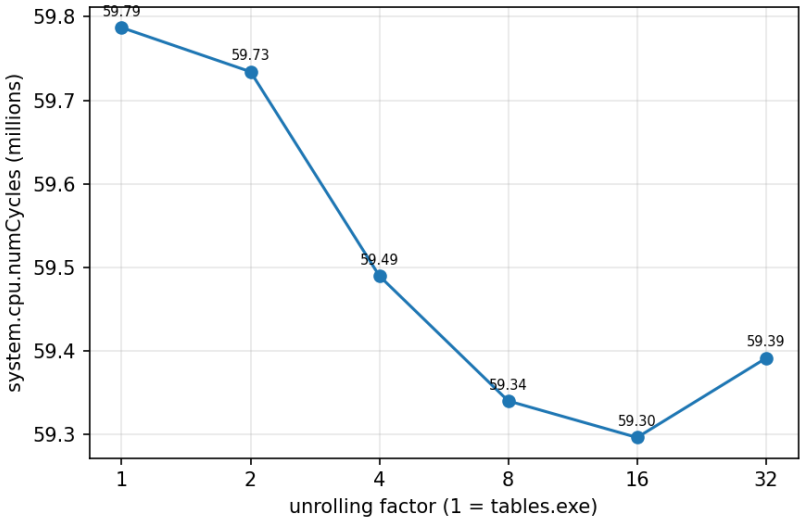
αντί για μια πρόσβαση στη DRAM.

3. Loop unrolling και μέγεθος L1D

Unrolling factor. Με την ίδια αρχιτεκτονική (8/8/128 KB) προσομοιώθηκαν οι εκδόσεις με unrolling factor 2 έως 32 (Πίνακας 6, Σχήμα 2).

Πίνακας 6. Κύκλοι εκτέλεσης ανά unrolling factor (L1I = L1D = 8 KB, L2 = 128 KB).

Unrolling factor	1 (`tables.exe`)	2	4	8	16	32
Κύκλοι (εκατ.)	59,79	59,73	59,49	59,34	**59,30**	59,39



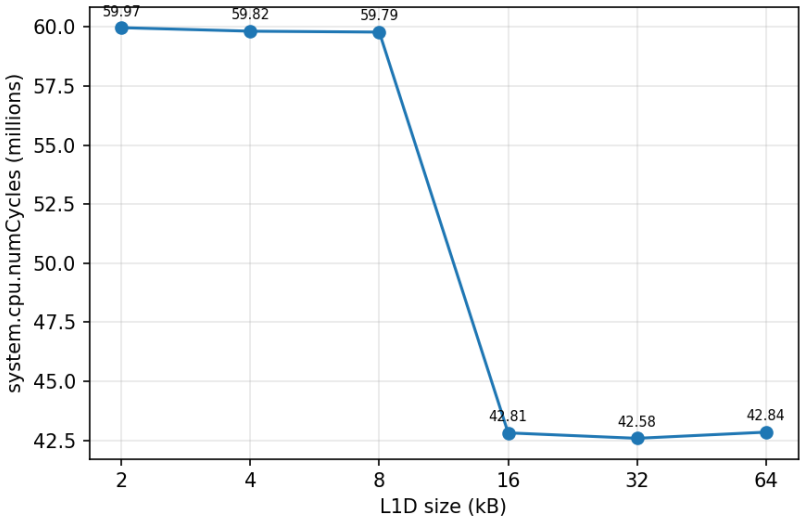
Σχήμα 2. Κύκλοι εκτέλεσης συναρτήσει του unrolling factor.

Το unrolling μειώνει τους κύκλους, αλλά μόνο κατά 0,8% στην καλύτερη περίπτωση (UF = 16). Το όφελός του προέρχεται από το overhead του βρόχου: με `-O0` ο μετρητής `i` βρίσκεται στο stack και σε κάθε επανάληψη γίνεται load, αύξηση, store και σύγκριση. Με UF = 16 οι προσβάσεις στην L1D πέφτουν από 4,44 σε 4,06 εκατομμύρια. Ο χρόνος όμως καθορίζεται από τα L1D misses (περίπου 817 χιλιάδες), τα οποία το unrolling δεν επηρεάζει. Για UF = 32 οι κύκλοι αυξάνονται ξανά. Ο κώδικας μεγαλώνει (7,3 KB στο `tables_uf32.exe` έναντι 1,9 KB στο `tables.exe`) και επιβαρύνει την L1I, χωρίς να μένει overhead βρόχου για να εξοικονομηθεί. Η επίδραση αυτή φαίνεται πολύ πιο έντονα με μικρότερη L1I στο ερώτημα 4.

Μέγεθος L1D. Με `tables.exe`, L1I = 8 KB και L2 = 128 KB:

Πίνακας 7. Κύκλοι εκτέλεσης και στατιστικά της L1D ανά μέγεθος L1D.

L1D	2 KB	4 KB	8 KB	16 KB	32 KB	64 KB
Κύκλοι (εκατ.)	59,97	59,82	59,79	**42,81**	42,58	42,84
L1D misses	824 278	–	816 756	110 454	–	101 993
L1D miss rate	18,6%	–	18,4%	2,5%	–	2,3%



Σχήμα 3. Κύκλοι εκτέλεσης συναρτήσει του μεγέθους της L1D.

Η καμπύλη δεν μειώνεται σταδιακά. Μένει σχεδόν σταθερή έως τα 8 KB, πέφτει απότομα κατά 28% στα 16 KB και μετά σταθεροποιείται ξανά. Η εξήγηση δεν είναι η χωρητικότητα: τα δεδομένα είναι 3,2 MB, πολύ περισσότερα από οποιαδήποτε από αυτές τις caches. Πρόκειται για **conflict misses**.

Σε syscall emulation mode το gem5 δίνει φυσικές σελίδες τη στιγμή της πρώτης πρόσβασης. Ο βρόχος αρχικοποίησης αγγίζει εναλλάξ τα `x1[i]`, `x2[i]`, `x3[i]` και `y[i]`, οπότε οι σελίδες των τεσσάρων πινάκων καταλήγουν διαδοχικές στη φυσική μνήμη, σε απόσταση 4 KB η μία από την άλλη. Οι εικονικές διευθύνσεις, που μετρήθηκαν μέσα στο gem5, απέχουν 0xC4000 και έχουν όλες offset 0x010 μέσα στη σελίδα. Σε μια 2-way cache μεγέθους C κάθε way καλύπτει $C/2$ bytes, και διευθύνσεις που διαφέρουν κατά πολλαπλάσιο του $C/2$ πέφτουν στο ίδιο set.

Για L1D $\leq$ 8 KB το way είναι το πολύ 4 KB, άρα και οι τέσσερις πίνακες αντιστοιχίζονται στο ίδιο set. Κάθε επανάληψη χρειάζεται 4 γραμμές σε ένα set με 2 θέσεις, και κάθε πρόσβαση διώχνει μια γραμμή που θα χρειαστεί αμέσως μετά (thrashing). Το αποτέλεσμα είναι περίπου ένα miss ανά πρόσβαση σε πίνακα. Για L1D = 16 KB το way είναι 8 KB: τα `x1`, `x3` μοιράζονται ένα set και τα `x2`, `y` ένα άλλο, οπότε οι δύο γραμμές κάθε set χωράνε. Τα misses πέφτουν στις 110 χιλιάδες περίπου, πολύ κοντά στο ελάχιστο των compulsory misses (3,2 MB / 64 B = 50 000 γραμμές, μία φορά σε κάθε έναν από τους δύο βρόχους). Πάνω από τα 16 KB δεν υπάρχει περαιτέρω κέρδος, αφού δεν υπάρχει επαναχρησιμοποίηση πέρα από τη γραμμή.

Για αυτό το μοτίβο προσβάσεων λοιπόν η associativity και η στοίχιση των δεδομένων μετρούν περισσότερο από το καθαρό μέγεθος της cache. Συνέπεια του ίδιου φαινομένου είναι ότι για L1D $\geq$ 16 KB το L2 miss rate εκτοξεύεται (91,7% για 16 KB): στην L2 φτάνουν πια μόνο τα compulsory misses, τα οποία δεν μπορεί να εξυπηρετήσει.

4. Εξαντλητική αναζήτηση και Pareto front

Ο χώρος σχεδίασης αποτελείται από L1I, L1D $\in \{2, \ldots, 64\}$ KB, L2 $\in \{128, \ldots, 1024\}$ KB και unrolling factor $\in \{2, 4, 8, 16, 32\}$, δηλαδή $6 \cdot 6 \cdot 4 \cdot 5 = 720$ σημεία. Όλες οι προσομοιώσεις εκτελέστηκαν με script. Κάθε προσομοίωση γράφει σε δικό της output directory, ώστε να τρέχουν 10 παράλληλα, και το αποτέλεσμα αποθηκεύεται σε CSV.

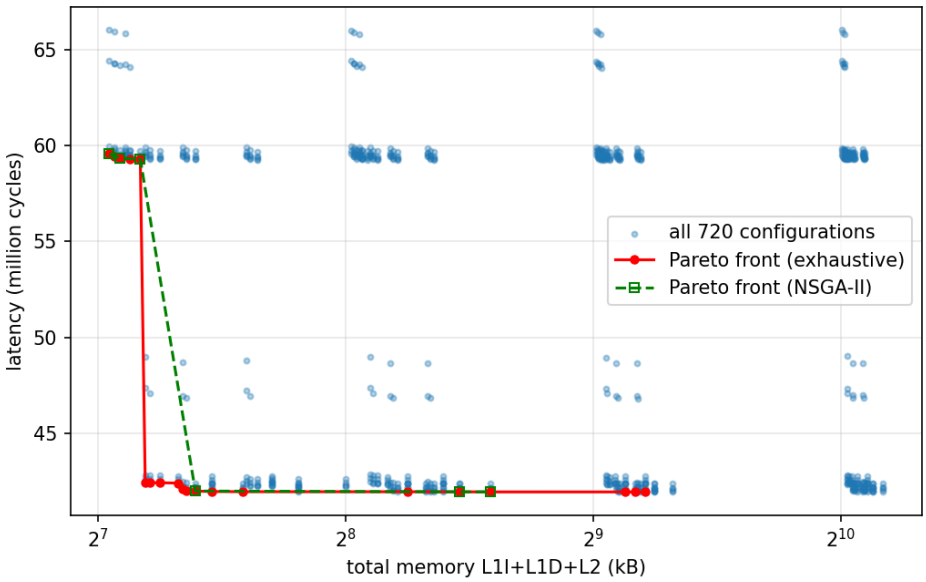
Χρόνος αναζήτησης. Οι 720 προσομοιώσεις ολοκληρώθηκαν σε 688 s με 10 παράλληλες εκτελέσεις. Το άθροισμα των επιμέρους χρόνων, δηλαδή ο χρόνος μιας σειριακής εξαντλητικής αναζήτησης, είναι **114 λεπτά**.

Η καλύτερη καθυστέρηση είναι 41 922 998 κύκλοι (16/64/512 KB, UF = 16, 592 KB συνολικά) και η

χειρότερη 66 060 573 (2/2/128 KB, UF = 32). Το Pareto front υπολογίστηκε με τη βιβλιοθήκη `paretoset` πάνω σε pandas dataframe με στήλες `totalMemoryKB` (L1I + L1D + L2) και `latencyCC`, ελαχιστοποιώντας και τα δύο. Αποτελείται από 20 σημεία (Πίνακας 8, Σχήμα 4).

Πίνακας 8. Pareto-optimal αρχιτεκτονικές της εξαντλητικής αναζήτησης.

L1I	L1D	L2	UF	Μνήμη (KB)	Κύκλοι
2 KB	2 KB	128 KB	8	132	59 581 198
2 KB	4 KB	128 KB	8	134	59 433 698
4 KB	4 KB	128 KB	16	136	59 343 511
4 KB	8 KB	128 KB	16	140	59 306 017
8 KB	8 KB	128 KB	16	144	59 296 471
2 KB	16 KB	128 KB	8	146	42 416 612
4 KB	16 KB	128 KB	16	148	42 409 832
8 KB	16 KB	128 KB	16	152	42 403 308
16 KB	16 KB	128 KB	16	160	42 387 518
2 KB	32 KB	128 KB	8	162	42 078 234
4 KB	32 KB	128 KB	16	164	41 969 420
8 KB	32 KB	128 KB	16	168	41 969 226
16 KB	32 KB	128 KB	16	176	41 945 801
32 KB	32 KB	128 KB	16	192	41 936 488
16 KB	32 KB	256 KB	16	304	41 933 039
64 KB	32 KB	256 KB	16	352	41 928 359
64 KB	64 KB	256 KB	16	384	41 925 024
16 KB	32 KB	512 KB	16	560	41 923 846
32 KB	32 KB	512 KB	16	576	41 923 699
16 KB	64 KB	512 KB	16	592	41 922 998



Σχήμα 4. Οι 720 αρχιτεκτονικές, το Pareto front της εξαντλητικής αναζήτησης (κόκκινο) και το Pareto front του γενετικού αλγορίθμου (πράσινο).

Η παράμετρος που καθορίζει την απόδοση είναι η L1D. Η μετάβαση σε L1D $\geq$ 16 KB, δηλαδή η αποφυγή των conflict misses του ερωτήματος 3, μειώνει την καθυστέρηση κατά 29% περίπου με κόστος

μόλις 2–14 KB μνήμης. Το «γόνατο» του front είναι η αρχιτεκτονική **L1I 2 KB / L1D 16 KB / L2 128 KB με UF = 8** (146 KB, 42,4 εκατ. κύκλοι). Από εκεί και πέρα κάθε επιπλέον μνήμη κερδίζει λιγότερο από 1,2%.

Η L2 αντίθετα σχεδόν δεν επηρεάζει: η καλύτερη καθυστέρηση για κάθε μέγεθος L2 διαφέρει λιγότερο από 0,05%. Τα δεδομένα διαβάζονται μία φορά, οπότε μια μεγαλύτερη L2 δεν έχει τι να επαναχρησιμοποιήσει, και γι' αυτό τα περισσότερα Pareto σημεία έχουν τη μικρότερη L2.

Τέλος, το unrolling αλληλεπιδρά με το μέγεθος της L1I, γιατί ένας βρόχος με μεγάλο unrolling δεν χωράει σε μικρή L1I (Πίνακας 9). Με L1I 2 KB το καλύτερο UF είναι το 8, ενώ με L1I $\geq$ 4 KB είναι το 16. Αυτό εξηγεί γιατί τα Pareto σημεία μικρής μνήμης έχουν UF = 8.

Πίνακας 9. Κύκλοι (εκατ.) ανά L1I και unrolling factor, για L1D = 32 KB και L2 = 128 KB.

L1I \ UF	2	4	8	16	32
2 KB	42,45	42,24	**42,08**	46,95	48,68
4 KB	42,40	42,19	42,03	**41,97**	46,85
$\geq$ 8 KB	42,39	42,18	42,01	**41,97**	42,16

5. Γενετικός αλγόριθμος

Το `genOptimizer.py` του μαθήματος εκτελέστηκε χωρίς αλλαγές. Χρησιμοποιεί τον NSGA-II της βιβλιοθήκης `pymoo` με πληθυσμό 10 και 5 απογόνους ανά γενιά, και το κριτήριο που τον σταματά εδώ είναι το μέγιστο των 5 γενεών. Έκανε **30 evaluations** ($10 + 4 \times 5$) σε **215 s**. Αυτό αντιστοιχεί στο 4,2% των προσομοιώσεων της εξαντλητικής αναζήτησης και σε περίπου 32 φορές μικρότερο χρόνο από τη σειριακή εκδοχή της (114 λεπτά). Το Pareto front που επέστρεψε φαίνεται στον Πίνακα 10.

Πίνακας 10. Pareto front του γενετικού αλγορίθμου και σύγκριση με το πραγματικό.

L1I	L1D	L2	UF	Μνήμη (KB)	Κύκλοι	Στο πραγματικό front;
2 KB	2 KB	128 KB	8	132	59 581 198	ναι
4 KB	4 KB	128 KB	16	136	59 343 511	ναι
8 KB	8 KB	128 KB	16	144	59 296 471	ναι
8 KB	32 KB	128 KB	16	168	41 969 226	ναι
32 KB	64 KB	256 KB	16	352	41 929 862	όχι
64 KB	64 KB	256 KB	16	384	41 925 024	ναι

Τα 5 από τα 6 σημεία είναι πράγματι Pareto-optimal. Το έκτο (32/64/256 KB) κυριαρχείται οριακά από την αρχιτεκτονική 64/32/256 KB, που έχει την ίδια συνολική μνήμη και 1 503 κύκλους λιγότερους. Ο αλγόριθμος βρήκε όμως μόνο 5 από τα 20 σημεία του πραγματικού front και, κυρίως, **έχασε το γόνατο**: κανένα σημείο του δεν έχει L1D = 16 KB. Έτσι, μεταξύ 144 και 168 KB το front του δίνει ακόμη 59 εκατ. κύκλους, ενώ με 146 KB επιτυγχάνονται ήδη 42,4 εκατ. (Σχήμα 4).

Ο γενετικός αλγόριθμος δίνει τη γενική μορφή του trade-off (μικρή και αργή αρχιτεκτονική απέναντι σε μεγάλη και γρήγορη) με ένα κλάσμα του κόστους, αλλά με 5 γενεές και πληθυσμό 10 δεν προλαβαίνει να συγκλίνει. Περισσότερες γενεές ή μεγαλύτερος πληθυσμός θα έκλειναν τη διαφορά με περισσότερες προσομοιώσεις. Εδώ η εξαντλητική αναζήτηση ήταν εφικτή μόνο επειδή κάθε προσομοίωση διαρκεί περίπου 9 s. Σε μεγαλύτερο χώρο σχεδίασης, ή με πιο αργές προσομοιώσεις, η ευρετική αναζήτηση είναι η μόνη ρεαλιστική επιλογή.

Παράρτημα: Κώδικας

Όλος ο κώδικας και τα δεδομένα των μετρήσεων βρίσκονται στο `https://github.com/Zajason/embedded_systems/tree/main/lab/Lab1`.

- `src/`: ο κώδικας της πλακέτας. Το `hal_entry.c` χρονομετρά όλες τις εκδόσεις. Ο αρχικός αλγόριθμος βρίσκεται στο `phods_baseline.h` και το v1 στο `phods_variants.h`. Από τον βελτιστοποιημένο κώδικα του `phods_impl.h` παράγονται τα v2–v4 και οι 16 εκδόσεις $B_x \times B_y$.
- `test/host_test.c`: έλεγχος ορθότητας όλων των εκδόσεων σε υπολογιστή.
- `tools/analyze.py`: μετατροπή των μετρήσεων της πλακέτας σε CSV και διαγράμματα.
- `part2/scripts/`: τα scripts του gem5 (ερωτήματα 1–5) και `part2/results/`: όλα τα αποτελέσματα σε CSV.